Contents

0.1 字符串哈希 . . . 2
0.1.1 有序字符串哈希 . . . 2
0.1.2 无序字符串哈希 . . . 3
0.1.3 二维有序字符串哈希 . . . 4
0.2 最小表示法 . . . 5
0.3 最小循环节 . . . 5
0.4 KMP . . . 6
0.5 Z 函数 (扩展 KMP) . . . 7
0.6 Manacher . . . 8
0.7 子序列自动机 . . . 9
0.8 回文自动机 . . . 11
0.9 后缀数组 . . . 13
0.10 AC 自动机 . . . 16
0.11 Trick . . . 18

std::vector M = {1610612741, 805306457, 402653189, 201326611, 100663319, 1000000007, 1000000009}; 随机两个模数

双哈希就是做两个哈希，同时判断

0.1 字符串哈希

注意哈希传入的字符串都是 $1-index$

Base 用 (rng()%100000)+256, P 从 M 中随机选

0.1.1 有序字符串哈希

我们考虑哈希函数为 $f(n)=\sum_{i=1}^{n} s[i]*Base^{n-i}$, 那么假设我们预处理出了所有的位置 i 对应的 $f(i)$,

那么 $[l,r]$ 的哈希值为 $f[r]-f[l-1]*Base^{r-l+1}$

```
struct StringHash
{
    int n;
    std::vector<int> Hash;
    std::vector<int> H;
    int Base, P;
    StringHash(const std::string &s, int Base, int P)
        : n(s.size()), Hash(n), H(n), Base(Base), P(P) // 默认字符串加了前导字符
    {
        Hash.assign(n, 0);
        H.assign(n, 0);
        Hash[0] = 1;
        for (int i = 1; i <= n - 1; ++i)
        {
            Hash[i] = Hash[i - 1] * Base % P;
        }
        for (int i = 1; i <= n - 1; ++i)
        {
            H[i] = (H[i - 1] * Base % P + s[i]) % P;
        }
    }
    int operator()(int l, int r)
    {
        return (H[r] - H[l - 1] * Hash[r - l + 1] % P + P) % P;
    }
};
```

0.1.2 无序字符串哈希

我们考虑哈希 j 函数为 $f(n)=\sum_{i=1}^{n}h[s[i]]$，其中 $h[x]$ 是一个随机的函数，只要保证 x 相同时 $h[x]$ 相同即可

```
struct StringHash
{
    int n;
    std::vector<int> Hash;
    std::vector<int> H;
    int Base, P;
    StringHash(std::vector<int> &a, int max, int Base, int P)
        : n(a.size()), Base(Base), P(P) // 无序哈希
    {
        Hash.assign(max + 1, 0);
        H.assign(n, 0);
        Hash[0] = 1;
        for (int i = 1; i <= max; ++i)
        {
            Hash[i] = Hash[i - 1] * Base % P;
        }
        for (int i = 1; i <= n - 1; ++i)
        {
            H[i] = (H[i - 1] + Hash[a[i]]) % P;
        }
    }
    int operator()(int l, int r)
    {
        return (H[r] - H[l - 1] + P) % P;
    }
};
```

0.1.3 二维有序字符串哈希

```cpp
using u64 = unsigned long long;
struct Hash
{
    int n, m;
    std::vector<u64> hX;//预处理行的 Base 次幂数组
    std::vector<u64> hY;//预处理列的 Base 次幂数组
    std::vector<std::vector<u64>> A;//哈希数组
    int P, Q;//两个 Base 分别为行和列

    Hash(const std::vector<std::vector<char>> &a, int n, int m, int P, int Q)
        : n(n), m(m), hX(n + 1), hY(m + 1), P(P), Q(Q)
    {
        A.assign(n + 1, std::vector<u64>(m + 1, 0));
        hX[0] = 1;
        hY[0] = 1;
        for (int i = 1; i <= n; ++i)
        {
            hX[i] = hX[i - 1] * P;
        }
        for (int i = 1; i <= m; ++i)
        {
            hY[i] = hY[i - 1] * Q;
        }
        for (int i = 1; i <= n; ++i)
        {
            for (int j = 1; j <= m; ++j)
            {
                A[i][j] = A[i][j - 1] * Q + A[i - 1][j] * P - A[i - 1][j - 1] * P * Q
                            + (a[i][j] - 'a' + 1);
            }
        }
    }

    u64 query(int x1, int y1, int x2, int y2)
    {
        return A[x2][y2] - A[x2][y1 - 1] * hY[y2 - y1 + 1] - A[x1 - 1][y2] * hX[x2 - x1 + 1]
                         + A[x1 - 1][y1 - 1] * hX[x2 - x1 + 1] * hY[y2 - y1 + 1];
    }
};
```

0.2 最小表示法

返回一个字符串字典序最小的循环同构串。

```
// 字符串可以先转化为元素类型为 char 的 vector 数组，
std::vector<int> minimalString(std::vector<int>& a) {
    int n = a.size();
    int i = 0, j = 1, k = 0;
    while (k < n and i < n and j < n) {
        if (a[(i + k) % n] == a[(j + k) % n]) {
            k++;
        } else {
            (a[(i + k) % n] > a[(j + k) % n] ? i : j) += k + 1;
            i += (i == j);
            k = 0;
        }
    }
    k = std::min(i, j);
    std::vector<int> ans(n);
    for (int i = 0; i < n; i++) { ans[i] = a[(i + k) % n]; }
    return ans;
}
// 直接返回字典序最小循环同构串
```

小 Trick：如果我们有一个字符串数组，我们希望按特定的顺序拼起来，使得其字典序最小，我们只需要重载排序函数，方法是 $a+b<b+a$ 其中 a,b 都是字符串类型

0.3 最小循环节

判断一个串是否有长度为 l 的循环节，我们只需要判断 $[l,r-d]$ 和 $[l+d,r]$ 处的串是否相同即可，其中 l 表示左端点，r 表示右端点。

如果要找到最小的循环节，我们考虑到答案一定是 len 的一个因数，并且由唯一分解定理一定可以表示为 $p_1^{a_1}*p_2^{a_2}...$，那么我们对 len 不断除去它的最小质因数（用欧拉筛处理），同时判断 ans 除去该质因数是否满足即可，在实现哈希函数后，复杂度为 $O(\sum a)$，即所有质因数的幂之和

```
// minPrimeFactor() 是通过欧拉筛 sieve 获取的最小质因数 minp
int findMinimalCycle(std::string s)
{
    int len = s.length();
    s = ' ' + s;
    int l = 1, r = len;
    StringHash hash(s, 13331, 998244353);
    int ans = len;
    while (len != 1)
    {
        int d = ans / minPrimeFactor(len);
        if (hash(l, r - d) == hash(l + d, r))
        {
            ans = d;
        }
        len /= minPrimeFactor(len);
    }
    return ans;
}
```

0.4 KMP

其中 $next[i]$ 表示字符串以 i 结尾的 和 s 前缀相同的长度小于 i 的最长长度

定义:

- 定义一个字符串 s 的 border 为 s 的一个**非 s 本身**的子串 t，满足 t 既是 s 的前缀，又是 s 的后缀。
- next 数组即为每个前缀 s 的最长 border t 的长度。

```
// 传入的字符串 0-base, 返回的数组 1-base
std::vector<int> kmp(std::string &s) {
    int n = s.size();
    std::vector<int> next(n + 1);
    // j 表示将要匹配的位置,
    // next[j] 表示回溯后将要匹配的位置
    // 已经匹配过的事 j-1 的长度
    for (int i = 1, j = 0; i < n; i++) {
        while (j && s[i] != s[j]) j = next[j];
        if (s[i] == s[j]) j++;
        next[i + 1] = j;
    }
    return next;
}

// 在 s 中找 t 的位置以及求 t 的 border
void solve(){
    string s,t;cin>>s>>t;
    vector<int> next = kmp(t);
    for(int i = 0,j = 0;i<s.size();i++){
        while(j && s[i] != t[j]) j = next[j];
        if(s[i] == t[j]) j++;
        if(j == t.size()) {
            cout<<i-j+1+1<<"\n"; // 0-base 位置 +1, 看情况
            j = next[j];
        }
    }
    for(int i = 1;i<=t.size();i++){
        cout<<next[i]<<" ";
    }
    cout<<"\n";
}
```

0.5 Z 函数 (扩展 KMP)

其中 $z[i]$ 数组表示 s 和 s 以 i 开头的后缀 $s.substr(i)$ 的最长公共长度 LCP

```
// 传入的字符串 0-base, 返回的数组 1-base
std::vector<int> Zfunc(const std::string &s) {
    int n = s.size();
    std::vector<int> z(n + 1);
    z[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i <= r) z[i] = std::min(z[i - l], r - i + 1);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
    }
    for (int i = n; i >= 1; i--) z[i] = z[i - 1];
    return z;
}
```

例题:

给定两个字符串 a, b，你要求出两个数组:

- b 的 z 函数数组 z，即 b 与 b 的每一个后缀的 LCP 长度。
- b 与 a 的每一个后缀的 LCP 长度数组 p。

对于一个长度为 n 的数组 a，设其权值为 $xor_{i=1}^{n} i \times (a_i + 1)$。

```
void solve1(){
    string a,b;cin>>a>>b;
    int la = a.size(),lb = b.size();
    b += "$"+a;
    vector<int> z = Zfunc(b);
    int ans1 = lb+1,ans2 = 0;
    for(int i = 2;i<=lb;i++) ans1 ^= i*(z[i]+1);
    for(int i = lb+2,j = 1;i<=la+lb+1;i++,j++) ans2 ^= j*(z[i]+1);
    cout<<ans1<<"\n"<<ans2<<"\n";
}
void solve2(){
    string a,b;cin>>a>>b;
    vector<int> z = Zfunc(b);
    int la = a.size(),lb = b.size();
    vector<int> p(la+1);
    for(int i = 0,l = 0,r = -1;i<la;i++){
        // z[i-l+1] 中加一是因为 z 整体右移了一位
        if(i<=r) p[i] = min(z[i-l+1],r-i+1);
        while(p[i]<lb and i+p[i]<la and b[p[i]] == a[i+p[i]]) p[i]++;
        if(i+p[i]-1>r) l = i,r = i+p[i]-1;
    }
    for(int i = la;i>=1;i--) p[i] = p[i-1];
    int ans1 = 0,ans2 = 0;
    for(int i = 1;i<=b.size();i++) ans1 ^= i*(z[i]+1);
    for(int i = 1;i<=a.size();i++) ans2 ^= i*(p[i]+1);
    cout<<ans1<<"\n"<<ans2<<"\n";
}
```

0.6 Manacher

其中 $r[i]$ 以位置 i 为中心的最长回文半径, 例如对于 $\#\mathrm{b}\#\mathrm{a}\#\mathrm{b}\#$，$i$ 为 a 的位置，此时 $r[i]=4$, 对于 $\#\mathrm{a}\#\mathrm{a}\#$，$i$ 是中间那个 $\#$ 的位置, 此时 $r[i]=3$

字符 s 中最大的回文串长度为 $max\{r[i]-1\}$

```
// 传入的 s 为 0-base
std::vector<int> manacher(std::string s) {
    std::string t = "#";
    for (auto c: s) {
        t += c;
        t += '#';
    }
    int n = t.size();
    std::vector<int> r(n);
    for (int i = 0, j = 0; i < n; i++) {
        if (2 * j - i >= 0 && j + r[j] > i) {
            r[i] = std::min(r[2 * j - i], j + r[j] - i);
        }
        while (i - r[i] >= 0 && i + r[i] < n && t[i - r[i]] == t[i + r[i]]) {
            r[i] += 1;
        }
        if (i + r[i] > j + r[j]) {
            j = i;
        }
    }
    return r;
}
// 使用 manacher 构造 r 数组时传入的 s 为 0-base
// 调用时 l, r 为 1-base
bool isPalindrome(std::vector<int> &R, int l, int r) {
    int p = (l + (r - l + 1) / 2) * 2;
    if ((r - l + 1) % 2 == 0) p--;
    return R[p] >= (r - l + 1);
}
// 对于 manacher 返回的 r 数组，询问位置 i 对应回文子串在原串中的端点 l,r
// 返回的 l, r 为 1-base
// 返回的 l > r 表示 i 为 # 且 i 两侧对应的字符不相等
pair<int,int> queryLR(int i,const vector<int>& R){
    int len = R[i] - 1;
    // (i - len) 是回文串在 t 中的起始位置
    // 除以 2 再加 1 正好对应 1-base 的起点
    int l = (i - len) / 2 + 1;
    int r = l + len - 1;
    return {l,r};
}
```

0.7 子序列自动机

子序列自动机即预处理出所有的 $next[i][c]$, 其中 i 表示位置，c 表示字符集，朴素做法的复杂度为 $O(nc)$

```
template <int Z, char Base>
struct SubSequenceAutomaton // 每个位置后面每个字母第一次出现的位置
{

    std::vector<std::array<int, Z>> next;
    int n;
    SubSequenceAutomaton(std::string s) : n(s.size() - 1) // 默认 s 是加过前缀空字符的
    {
        next.resize(n + 1);

        for (int i = 0; i < Z; ++i)
        {
            next[n][i] = n + 1;
        }
        for (int i = n - 1; i >= 0; --i)
        {
            next[i] = next[i + 1];
            next[i][s[i + 1] - Base] = i + 1;
        }
    }
    int getNextPos(int pos, char c)
    {
        return next[pos][c - Base];
    }
};
```

不难发现子序列自动机中每个位置只会有一个字符的值被修改，这启发我们使用主席树优化，这适用于字符集 c 很大时，同样的我们的查询也会由 $O(1)$ 变成为 $O(logn)$

```
struct PresidentTree
{
    int idx = 0;
    int n;
    std::vector<int> root, lc, rc;
    std::vector<int> sum;

    void pushup(int u)
    {
        sum[u] = sum[lc[u]] + sum[rc[u]];
    }
    PresidentTree(const std::vector<int> &a, int len)
        : n(a.size()), root(len + 1), lc(len * 40), rc(len * 40), sum(len * 40)
    { // 传入的 a 实际上是值域
        std::function<void(int &, int, int)> build = [&](int &u, int l, int r)
        {
            u = ++idx;
            if (l == r)
            {
                sum[u] = a[l];
                return;
            }
            int mid = l + r >> 1;
            build(lc[u], l, mid);
            build(rc[u], mid + 1, r);
            pushup(u);
        };
        build(root[len], 1, n - 1); //  倒着建树
    }
    void insert(int &u, int v, int l, int r, int pos, int x) // 要插入新版本 所以 idx 必须 ++
    {
        u = ++idx; // 一定是直接加 不用判断是不是 0
        lc[u] = lc[v];
        rc[u] = rc[v];
        sum[u] = sum[v];
        if (l == r)
        {
            sum[u] = x;
            return;
        }
        int mid = l + r >> 1;
        if (pos <= mid)
        {
            insert(lc[u], lc[v], l, mid, pos, x);
        }
        else
        {
            insert(rc[u], rc[v], mid + 1, r, pos, x);
        }
        pushup(u);
    }
    void insert(int now, int pre, int pos, int x)
    {
        insert(root[now], root[pre], 1, n - 1, pos, x);
```

```
    }
    int query(int u, int l, int r, int pos)
    {
        if (l == r)
        {
            return sum[u];
        }
        int mid = l + r >> 1;
        if (pos <= mid)
        {
            return query(lc[u], l, mid, pos);
        }
        else
        {
            return query(rc[u], mid + 1, r, pos);
        }
    }
    int query(int t, int pos)
    {
        return query(root[t], 1, n - 1, pos);
    }
};
// 子序列自动机
struct SubsequenceAutomaton
{
    const std::vector<int> &val;
    PresidentTree Tree;
    int n;
    SubsequenceAutomaton(const std::vector<int> &a, std::vector<int> &init)
        : val(a), n(a.size() - 1), Tree(init, a.size() - 1)
    {
        for (int i = n - 1; i >= 0; --i)
        {
            Tree.insert(i, i + 1, val[i + 1], i + 1); // 更新子序列自动机
        }
    }
    int query(int pos, int val) // pos 后面第一个 val 的位置
    {
        return Tree.query(pos, val);
    }
};
```

0.8 回文自动机

为了分别表示奇回文串和偶回文串，回文自动机是两棵树加 $fail$ 指针的结构。我们不妨设左树维护偶回文串，根节点为 0，右树维护奇回文串，根节点为 -1。特别的，的 $fail$ 指针指向 1

自动机上每个节点都代表一个回文串，如果 i 点代表的串为 S，那么点 i 经字符 c 的边到达的节点为 cSc，其中 $fail$ 指向最长的回文后缀。

```
template <int Z, char Base>
struct Pam
{
    std::vector<std::array<int, Z>> ch;   // 转移数组，存储每个节点的子节点
    std::vector<int> fail, len, dep, cnt; // fail 指针数组，回文长度数组，深度数组，计数器
    std::string s;                        // 存储添加的字符

    int cur = 0, tot = 1; // 当前节点和总节点数，初始有两个节点：0（偶长度根）和 1（奇长度根）
```

```
// 构造函数，初始化数组大小
Pam(int n) : ch(n + 2), fail(n + 2), len(n + 2), dep(n + 2), cnt(n + 2)
{
    fail[0] = 1; // 偶长度根的 fail 指向奇长度根
    len[1] = -1; // 奇长度根的回文长度为-1，作为特殊标记
}

// 用字符串初始化的构造函数
Pam(const std::string &s) : Pam(s.size())
{
    for (int i = 0; i < s.size(); i++)
    {
        add(i, s[i]); // 逐个添加字符
    }
}

// 节点计数函数，可用于统计每个回文子串的出现次数
void assign(int cur, int id)
{
    cnt[cur]++;
}

// 获取当前节点的 fail 指针
int getfail(int x, int i)
{
    // 循环找到第一个满足条件的 fail 节点
    while (i - len[x] - 1 < 0 or s[i - len[x] - 1] != s[i])
    {
        x = fail[x];
    }
    return x;
}

// 添加字符到 PAM
void add(int i, char c)
{
    c -= Base;       // 字符映射到 0-Z 范围内
    s.push_back(c); // 添加到字符串

    int v = getfail(cur, i); // 获取当前节点的 fail 节点

    // 如果该转移不存在，则创建新节点
    if (!ch[v][c])
    {
        fail[++tot] = ch[getfail(fail[v], i)][c]; // 设置新节点的 fail 指针
        ch[v][c] = tot;                           // 设置转移
        len[tot] = len[v] + 2;                    // 新节点的回文长度
        dep[tot] = dep[fail[tot]] + 1;            // 新节点的深度
    }

    cur = ch[v][c]; // 更新当前节点
    assign(cur, i); // 更新计数
}

// 获取 fail 树的邻接表表示，根节点为 1
auto getfailTree() const
```

```
    {
        std::vector e(tot + 1, std::vector<int>());
        for (int i = 0; i <= tot; i++)
        {
            if (i != 1) // 跳过根节点
            {
                e[fail[i]].emplace_back(i); // 添加边
            }
        }
        return e;
    }
};
```

0.9 后缀数组

后缀数组的三个核心函数：$sa, rk, height$，$sa[i]$ 表示排名为 i 的后缀编号，$rk[i]$ 表示编号为 i 的后缀排名，不难发现 sa 和 rk 互为反函数。$height[i] = lcp(sa[i], sa[i-1]))$，即两个排名相邻的后缀的 lcp。

我们利用倍增 + 计数排序可以在 $O(nlogn)$ 的时间内算出 $sa, rk, height$

重要性质（常见用法）：

两个后缀的最长公共前缀：$LCP(sa[i], sa[j]) = min_{x=i+1}^{j} height[x]$

不同子串数目 $num = n * (n+1)/2 - \sum_{i=2}^{n} height[i]$，即所有子串数目减去重复子串数目

查找子串及其出现次数：可以在 $rank$ 数组上二分，复杂度 $logn * |S|$，

最长重复子串（重叠或不重叠）：利用 $height$ 可以直接求得

字符串匹配问题：我们把模式串和匹配串用空格拼在一起，利用 $rank$ 即可。

最长公共子串：仍然考虑把串拼在一起，利用 $height$ 求解

```
struct SuffixArray
{
    const int n; // 字符串长度
    std::vector<int> sa, rk, h; // 后缀数组、排名数组、高度数组

    template <class T>
    SuffixArray(const T &s): n(s.size()), sa(n), rk(n), id(n), tmp(n)
    {
        if (n == 0) return;
        // 初始化 id 数组为 0~n-1（用于排序下标）
        std::iota(begin(id), end(id), 0);
        // 初始排名为字符 ASCII 值
        for (int i = 0; i < n; i++) {
            rk[i] = s[i];
        }
        countSort(); // 第一次计数排序

        // 倍增法主循环（按双关键字排序）
        for (int w = 1;; w <<= 1) {
            // 处理长度为 w 的后缀，构建次关键字（起始位置 >=w 的后缀，次关键字为前 w 位的排名）
            std::iota(begin(id), begin(id) + w, n - w); // 初始化次关键字下标（末尾 w 个元素优先）
            for (int i = 0, p = w; i < n; i++) { // 收集主关键字下标（起始位置 <w 的后缀，主关键字为当前排名）
                if (sa[i] >= w) { // sa[i] 是当前已排序的后缀起始位置
                    id[p++] = sa[i] - w; // 次关键字为 sa[i]-w 位置的排名（主关键字 +w 的排名）
                }
            }
            countSort(); // 按双关键字排序
```

```
            oldrk = rk; // 保存上一轮排名

            // 重新计算新排名
            rk[sa[0]] = 0;
            for (int i = 1, p = 0; i < n; i++) {
                // 判断两个后缀是否相等（考虑长度 w 的双关键字）
                rk[sa[i]] = equal(sa[i], sa[i - 1], w) ? p : ++p;
            }

            // 所有排名唯一时终止（已完成所有后缀排序）
            if (rk[sa.back()] + 1 == n) {
                break;
            }
        }

        calcHeight(s); // 计算高度数组
    }

private:
    std::vector<int> oldrk, id, tmp, cnt; // 临时数组（旧排名、下标数组、临时存储、计数数组）

    template <class T>
    inline void calcHeight(const T &s)
    {
        h.assign(n, 0); // 初始化高度数组
        for (int i = 0, k = 0; i < n; i++) {
            if (rk[i] == 0) {  // 排名为 0 的后缀是最大的，无前驱后缀
                k = 0;
                continue;
            }
            if (k) --k;  // 避免 k 为 0 时减 1（等价于 k = k > 0 ? k-1 : 0）
            // 计算后缀 i 与前一名后缀（sa[rk[i]-1]）的最长公共前缀
            int j = sa[rk[i] - 1];
            while (i + k < n && j + k < n && s[i + k] == s[j + k]) {// 逐个字符比较
                ++k;
            }
            h[rk[i]] = k; // 高度数组存储排名为 rk[i] 的后缀与前一名的 LCP
        }
    }

    // 计数排序（用于双关键字排序）
    inline void countSort()
    {
        int m = *std::max_element(begin(rk), end(rk)); // 获取当前最大排名值
        cnt.assign(m + 1, 0); // 计数数组大小为最大排名 +1
        // 统计每个排名的出现次数（tmp 存储当前下标的排名）
        for (int i = 0; i < n; i++) {
            cnt[tmp[i] = rk[id[i]]] += 1;
        }
        // 计算前缀和（确定排序后位置）
        for (int i = 1; i < cnt.size(); i++) {
            cnt[i] += cnt[i - 1];
        }
        // 逆序填充 sa 数组（稳定排序）
        for (int i = n - 1; i >= 0; i--) {
```

```
            sa[--cnt[tmp[i]]] = id[i]; // 根据排名和计数确定位置
        }
    }

    // 判断两个后缀的前 w 个字符是否相等（基于旧排名）
    inline bool equal(int x, int y, int w)
    {
        // 获取 x 和 y 位置的后缀，长度为 w 的双关键字排名
        int rkx = (x + w < n ? oldrk[x + w] : -1); // 若越界，视为排名-1（小于任何有效排名）
        int rky = (y + w < n ? oldrk[y + w] : -1);
        // 主关键字（前 w 长度）和次关键字（后 w 长度）均相等时视为相等
        return oldrk[x] == oldrk[y] && rkx == rky;
    }
    /**
     * sa[i] -> 第 i 小的后缀（起始下标）
     * rk[i] -> 后缀 i 的排名（0~n-1）
     */
};
```

0.10 AC 自动机

AC 自动机即在 $trie$ 树的基础上补成一个 $trie$ 有向图，并求出 $fail$ 指针，$trie$ 图的意义在于，它借助 $fail$ 指针补上了能继续匹配下去的边，$fail$ 指针的意义是指向最长的相等后缀，如果我们当前 $ch[p][c]$ 没有出边，我们就考虑 $ch[fail[p]][c]$ 即可。

$fail$ 树的意义在于，如果节点 P 出现过那么所有的 $fail$ 也将出现，注意我们的 $fail$ 树和 $trie$ 树共用所有点但不共用边，换句话说，$fail$ 树独立在 $trie$ 图以外

我们考虑一个串在 $trie$ 图上走的意义，首先建图方式保证了我们走图的最优性，什么叫最优性呢，也就是说在走不下去时会找到最长的后缀继续走下去。我们每走到一个点，该点在 $fail$ 树上的每个祖先都会出现一次，由于最优性保证了不重不漏。因此多模式匹配的做法就是，先在 $trie$ 图上跑完整个串，对于每个到达的节点都标记一次贡献。然后我们利用 dfs 或者 $toposort$ 的方式自下而上合并，把所有贡献合并给其祖先即可，此时每个节点的权值就代表了它出现的次数。

传入的 $vectorstd::string\&s$ 为 $0-base$

- Z: 字符集大小（例如小写字母就是 26）。
- Base: 字符集的起始偏移量（例如小写字母就是 'a'）。
- ch[u][i]: 转移函数。在 build() 之后，它实际上变成了 **Trie 图**，如果当前字符不存在，它会自动指向失配后能匹配的最长前缀节点。
- fail[u]: 失配指针。指向当前状态的最长后缀。
- ID[p]: 存储在节点 p 结尾的模式串索引（处理多个模式串相同的情况）。

```
template <int Z, char Base>
struct AcAutomaton
{
    std::vector<int> fail; // fail 指针
    std::vector<std::vector<int>> ch;
    std::vector<std::vector<int>> ID;
    // std::vector<vector<int>> ftr; 建 fail 链树
    int idx = 0;
    int sum = 0;
    int n;

    AcAutomaton(std::vector<std::string> &s) : n(s.size())
    {
        for (auto x : s)
        {
            sum += x.size();
        }
        ch.assign(sum + 1, std::vector<int>(Z, 0));
        fail.resize(sum + 1);    ftr.resize(sum + 1);
        ID.resize(sum + 1);
        for (int i = 0; i < n; ++i)
        {
            insert(i, s[i]);
        }
        build();
    }

    void insert(int id, std::string s)
    {
        int p = 0;
        for (auto j : s)
        {
            j -= Base;
            if (!ch[p][j])
            {
```

```
                ch[p][j] = ++idx;
            }
            p = ch[p][j];
        }
        ID[p].push_back(id);
    }

    void build()
    {
        std::queue<int> q;
        for (int i = 0; i < 26; ++i)
        {
            if (ch[0][i])
            {
                q.push(ch[0][i]);
            }
        }
        while (q.size())
        {
            int u = q.front();
            q.pop();
            // ftr[fail[u]].emplace_back(u); 建 fail 链树，可用作 dp 计数
            for (int i = 0; i < 26; ++i)
            {
                int v = ch[u][i];
                if (v)
                {
                    fail[v] = ch[fail[u]][i]; // fail 指针回跳边
                    q.push(v);
                }
                else
                {
                    ch[u][i] = ch[fail[u]][i]; // 横插边
                }
            }
        }
    }
};
```

例: 求每个模式串 s 在主串 t 中出现的次数

```
void solve(){
    int n;cin>>n;
    vector<string> s(n);
    for(int i = 0;i<n;i++){
        cin>>s[i];
    }
    AcAutomaton<26,'a'> ac(s);
    string t;cin>>t;
    vector<int> vis(ac.sum+10);
    int p = 0;
    for(int i = 0;i<size(t);i++){
        int c = t[i]-'a';
        p = ac.ch[p][c];
        ac.cnt[p]++;
    }
    auto &e = ac.ftr;
    int mx = 0;
    vector<int> ans(n);
    // 树形 DP
    auto dfs = [&](auto dfs,int u,int fa)->void{
        for(int v:e[u]){
            dfs(dfs,v,u);
        }
        ac.cnt[fa] += ac.cnt[u];
        for(int x:ac.ID[u]) {
            ans[x] += ac.cnt[u];
        }
    };
    dfs(dfs,0,0);
    for(int x:ans){
        cout<<x<<"\n";
    }
}
```

0.11 Trick